Relatório Técnico – Kaida Studio ProximSys

1. Introdução

O **Kaida Studio – ProximSys** é um sistema de monitoramento de proximidade desenvolvido com o objetivo de detectar obstáculos em tempo real, emitir alertas visuais e sonoros e registrar eventos críticos em banco de dados. O projeto integra hardware e software por meio da utilização de um sensor ultrassônico conectado a um Arduino, um backend desenvolvido em Python, uma interface web para monitoramento em tempo real e um banco de dados PostgreSQL para armazenamento de eventos relevantes.

---

2. Objetivo

O principal objetivo do sistema é monitorar continuamente a distância entre um sensor e um obstáculo, classificando o nível de risco de acordo com a proximidade detectada.
Além da exibição local das informações, o sistema disponibiliza os dados em uma interface web e registra situações críticas para posterior consulta e análise.

---

3. Tecnologias Utilizadas

Hardware

- Arduino Uno
- Sensor Ultrassônico HC-SR04
- Display LCD I2C 16x2
- Buzzer Ativo 5V
- Protoboard e cabos de conexão

Software

- Arduino IDE
- Python 3
- Flask
- PostgreSQL
- HTML5
- CSS3
- JavaScript
- VS Code

Requisitos Funcionais



RF01 — Medição contínua de distância
O sistema deve realizar a medição contínua da distância entre o sensor ultrassônico e o objeto detectado, durante toda a operação.
RF02 — Conversão da leitura em status operacional
O sistema deve converter cada leitura de distância em um status de proximidade previamente definido.
RF03 — Classificação em faixas de risco
O sistema deve classificar a distância em cinco faixas distintas: **TA_LIVRE**, **CUIDADO**, **MUITO_CUIDADO**, **PERIGO** e **COLISAO_EMINENTE**.
RF04 — Exibição da distância no display LCD
O sistema deve apresentar no display LCD a distância medida em tempo real.
RF05 — Exibição do status no display LCD
O sistema deve apresentar no display LCD o status correspondente ao nível de risco identificado.
RF06 — Emissão de alerta sonoro progressivo
O sistema deve acionar um buzzer ativo com intensidade ou frequência de alerta progressiva conforme a redução da distância entre o sensor e o obstáculo.
RF07 — Desativação do alerta sonoro em situação segura
O sistema deve manter o buzzer desligado quando a leitura indicar situação de segurança.
RF08 — Envio de dados pela interface serial
O Arduino deve enviar os dados de distância e status para o computador por meio da comunicação serial USB.
RF09 — Recepção dos dados no backend
O backend deve receber, interpretar e processar os dados enviados pelo Arduino via serial.
RF10 — Disponibilização da leitura atual por API
O sistema deve expor uma rota de API capaz de retornar a leitura atual do sensor em formato JSON.
RF11 — Atualização automática do frontend
O sistema deve atualizar automaticamente os valores exibidos no navegador, sem necessidade de recarregar manualmente a página.
RF12 — Exibição de gauge/medidor visual
O frontend deve exibir um medidor visual da proximidade, podendo assumir formato de barra ou velocímetro.
RF13 — Alternância de estilo do gauge
O sistema deve permitir a alternância entre estilos visuais do gauge, conforme interação do usuário.
RF14 — Exibição de status legível na interface web
O sistema deve converter os códigos internos do sensor em textos amigáveis para exibição no frontend.
RF15 — Exibição de histórico de eventos
O sistema deve apresentar no navegador uma área de histórico com os eventos relevantes do sensor.
RF16 — Registro de eventos críticos em banco de dados
O sistema deve armazenar no banco de dados apenas eventos classificados como críticos, especialmente **PERIGO** e **COLISAO_EMINENTE**.
RF17 — Evitar duplicidade de registros
O sistema deve impedir a gravação repetida de um mesmo evento crítico enquanto o status permanecer inalterado.
RF18 — Consulta do histórico persistido
O sistema deve permitir a consulta, no frontend, dos eventos armazenados no banco de dados.
RF19 — Exibição de estatísticas do histórico
O sistema deve apresentar informações consolidadas, como total de eventos registrados e data/hora da última colisão.
RF20 — Exportação de histórico
O sistema deve permitir a exportação dos dados do histórico em formato CSV.
RF21 — Identificação visual da aplicação
O sistema deve exibir o nome **Kaida Studio - ProximSys** como identidade principal da aplicação.
RF22 — Exibição da marca no frontend
O sistema deve exibir a logo da Kaida Studio no topo da interface e também de forma sutil como elemento de fundo.

RF23 — Continuidade de funcionamento
O sistema deve operar continuamente enquanto o Arduino estiver conectado e o backend estiver em execução.
RF24 — Tratamento de leituras inválidas
O sistema deve ignorar leituras inválidas ou incompletas sem interromper a operação geral.
RF25 — Registro de data e hora dos eventos
Cada evento salvo no banco de dados deve conter data e hora da ocorrência.

Requisitos Não Funcionais

RNF01 — Execução local
O sistema deve operar em ambiente local, sem dependência de servidor remoto.
RNF02 — Baixa latência de atualização
A interface web deve refletir as alterações do sensor com atraso mínimo perceptível.
RNF03 — Resposta rápida do alerta sonoro
O alerta sonoro deve reagir de forma imediata à alteração do nível de proximidade.
RNF04 — Estabilidade da leitura serial
A aplicação deve suportar a leitura contínua da serial sem travamentos frequentes ou perda de comunicação.
RNF05 — Robustez contra falhas parciais
Falhas pontuais na leitura do sensor, no envio serial ou no salvamento de dados não devem encerrar o sistema inteiro.
RNF06 — Organização modular
O código deve estar organizado em módulos separados para Arduino, backend, frontend e banco de dados.
RNF07 — Facilidade de manutenção
O sistema deve permitir alterações futuras em textos, cores, faixas de distância e regras de alerta com baixo impacto estrutural.
RNF08 — Interface intuitiva
A interface web deve ser clara, legível e fácil de compreender por usuários não técnicos.
RNF09 — Identidade visual consistente
A aplicação deve manter uma identidade visual homogênea, com paleta de cores preta e vermelha, alinhada à marca.
RNF10 — Legibilidade visual
Textos, indicadores e tabelas devem manter contraste suficiente para leitura confortável.
RNF11 — Responsividade da interface
O frontend deve manter boa apresentação em diferentes resoluções de tela, especialmente em notebooks e monitores comuns.
RNF12 — Persistência de dados
Os eventos críticos devem permanecer armazenados no banco de dados mesmo após reinicialização do backend.
RNF13 — Integridade dos registros
Os dados gravados no banco devem refletir fielmente os eventos críticos detectados pelo sistema.
RNF14 — Redução de poluição do banco
O sistema deve evitar armazenar leituras repetitivas ou irrelevantes, preservando a utilidade do histórico.
RNF15 — Separação entre apresentação e lógica
A lógica de negócio deve permanecer no backend, enquanto o frontend deve se limitar à exibição e interação.
RNF16 — Compatibilidade com ambiente Windows
O sistema deve funcionar em ambiente Windows com Python, Flask, Arduino IDE e PostgreSQL.

RNF17 — Compatibilidade com navegador moderno
A aplicação web deve ser acessível por navegadores contemporâneos, como Chrome, Edge e Firefox.
RNF18 — Segurança em ambiente local
O sistema não deve expor o banco de dados nem a interface de controle para a internet pública.
RNF19 — Baixo consumo de recursos
O sistema deve operar de maneira leve, sem exigir alto consumo de memória ou processamento em máquinas convencionais.
RNF20 — Escalabilidade estrutural
A arquitetura deve permitir a inclusão futura de novos sensores, novos tipos de alerta e novas rotas de API sem reescrita completa.
RNF21 — Confiabilidade da persistência
O salvamento de eventos no banco deve ocorrer apenas quando a operação de inserção for concluída com sucesso.
RNF22 — Rastreabilidade dos eventos
Cada evento crítico deve ser facilmente rastreável por meio de data, hora, distância e status.
RNF23 — Reatividade visual adequada
O gauge e os indicadores de status devem acompanhar a variação da distância de forma visualmente coerente.
RNF24 — Separação de responsabilidades
O Arduino deve cuidar da captura e sinalização local, o backend da lógica e persistência, e o frontend da visualização.

.

Requisitos adicionais

RA01 — Histórico seletivo e relevante
O sistema deve registrar somente eventos que representem risco real, garantindo que o histórico tenha valor analítico.
RA02 — Representação visual da gravidade
O sistema deve associar cores e intensidade visual ao grau de risco apresentado.
RA03 — Integração multicanal
O sistema deve apresentar a mesma informação em três canais distintos: LCD físico, interface web e alerta sonoro.
RA04 — Atualização contínua do painel
A interface web deve operar em tempo real ou quase em tempo real durante a execução do sistema.



Diagrama de Caso de Uso

